

Hardware Trojan Detection via attention based fusion of Graph Neural Networks and Large Language Models

Capstone Project Phase 2 (2 Credits) | ISA-2 Presentation
Project ID: N56 | Guide: Dr. Vaishnavi Sankar

Papers Covered:

- ① GNN-based HT Detection – Yasaei et al., 2022
- ② PoisonedGNN: Backdoor Attack on GNNs – Alrahis et al., 2023
- ③ CatBoost Label-Flipping Poisoning – Sharma et al., 2022
- ④ PEARL: LLM-based HT Detection – Kundu et al., 2025

TEAM COMPOSITION

Sl No.	Name	SRN (In Ascending order)	Photo
1	Rohit Patil	PES1UG23EC255	
2	Saher Fathima Sharf	PES1UG23EC263	
3	Samarth Somashekar	PES1UG23EC270	
4	Shreya Save	PES1UG23EC288	

Guide : Dr. Vaishnavi Sankar

- Introduction & Motivation
- Literature Survey — Summary Table (All 4 Papers)
- Paper ① GNN-based HT Detection: Key Points, Analysis, Research Gaps, Future Scope
- Paper ② PoisonedGNN Backdoor Attack: Key Points, Analysis, Research Gaps, Future Scope
- Paper ③ CatBoost Label-Flipping Defense: Key Points, Analysis, Research Gaps, Future Scope
- Paper ④ PEARL LLM-based Detection: Key Points, Analysis, Research Gaps, Future Scope
- Comparative Analysis Across All 4 Papers
- Consolidated Research Gaps & Future Directions
- References

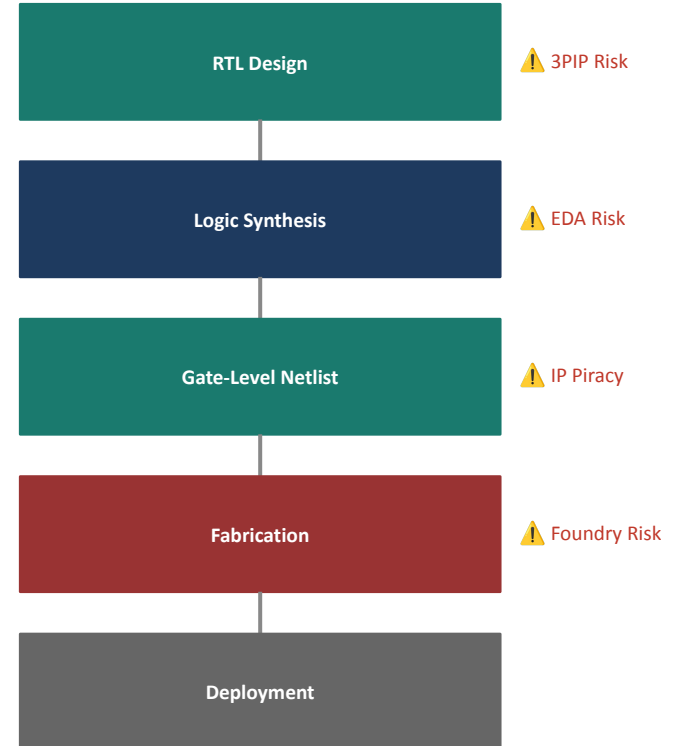
Introduction & Motivation

Hardware Trojans are malicious components embedded in integrated circuits (ICs) that cause abnormal or unintended behavior. They are often extremely small, difficult to detect. Once activated, they can:

- Leak confidential data
- Degrade chip performance
- Bypass security protections in military-grade hardware

As chip companies increasingly depend on third-party reusable hardware blocks, it becomes very important to detect Trojans early at RTL or gate-level before fabrication, so the entire chip supply chain remains trustworthy.

At the RTL design stage, untrusted third-party IP blocks may contain malicious logic. During logic synthesis, compromised EDA tools may manipulate the design. At the gate-level netlist stage, IP piracy or hidden structural changes may occur. During fabrication, untrusted foundries may also insert unauthorized modifications.



Literature Survey

Summary Overview

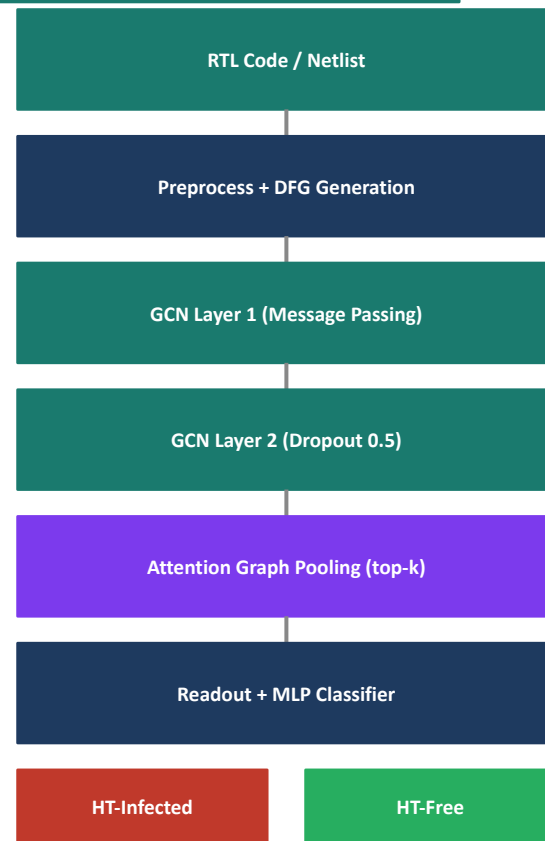
All 12 Papers at a Glance

#	Paper	Approach	Key Result	Main Gap
01	GNN-Based Detection	Converts netlist to Data Flow Graph (DFG). Uses GCN layers	RTL Recall: 97% Gate-level Recall: 84% Runtime: 21.1	Structural reasoning only — ignores semantic information in RTL code.
02	Node-wise GNN	Node-wise GNN on DFGs for fine-grained Trojan localization.	Recall: 96% Runtime: ~10–300 ms/circuit	Relies mainly on structural graph features, ignoring RTL semantics .
03	GNN + Structural Checking	HDL → directed graph with Bag-of-Words embeddings. Implements GCN, GraphSAGE.	Recall: 97% Runtime: ~10–500 ms/circuit	Design-level detection only — cannot identify exact malicious signals.
04	LLM Reasoning	Evaluates LLM reasoning with domain-specific datasets and structured prompt variations	Benchmarks LLM accuracy and robustness across diverse industries	LLMs cannot effectively analyze structural relationships. Limited capabilities
05	LLM-based Detection	Detects Trojans from HDL semantics using pretrained LLMs with strong zero-shot and few shot	Strong zero-shot and few-shot Trojan detection performance	Breaks under RTL structural modifications. Cannot detect Trojans that are obfuscated
06	LLM + Code Generation	Iterative LLM generation + compiler-driven refinement to improve Verilog code	Improved Verilog code correctness through iterative refinement	Lacks deep hardware interpretation. Cannot identify malicious logic or conditions
07	Graph + Transformer Fusion	GNN message passing + transformer attention for structural-semantic fusion	Strong structural-semantic fusion on text-attributed natural language	Tested only on natural-language graphs. Lacks hardware-aware priors.
08	Feature Engineering	Hand-engineered structural features (fanin/fanout, controllability) fed	Node-level Trojan gate classification using structural features	Features are manually defined and cannot adapt to unseen Trojans.
09	XAI for HT Detection	XAI applied on top of structural netlist analysis to explain why circuit regions behave certain way	Explainability of structural Trojan detection decisions	Explainability is post-hoc and structural-only. No semantics, no fusion
10	LLM + Formal Verification	LLMs auto-generate formal properties for RTL designs to reduce manual specifications	Reduced manual effort for formal verification specification	RTL-only. LLM acts as an assistant, not a detector. No GNN, no gate-level
11	XAI Benchmarking	Benchmarks multiple XAI methods across existing HT detectors to evaluate explanations	Comparative evaluation of XAI methods across HT detectors	No new model proposed. All methods are single-modality with post-hoc explanation
12	Transfer Learning	Reviews how pretrained knowledge boosts performance in data-scarce settings.	Demonstrates performance gains from pretrained knowledge	Provides no hardware-specific insights or evaluations. Limited directiveness

[1] R. Yasaei, L. Chen, S.-Y. Yu, M. A. Al Faruque, "Hardware Trojan Detection using Graph Neural Networks," IEEE TCAD, arXiv:2204.11431, 2022.

KEY POINTS:

- Proposes golden reference-free HT detection for both RTL and gate-level netlists
- Converts hardware designs into Data Flow Graphs (DFGs) capturing signal dependencies
- Applies attention-based graph pooling (top-k filtering) to focus on critical circuit nodes
- Graph embedding via max-pooling readout → MLP classifier → binary: HT-infected / HT-free
- Achieves 97% Recall on RTL in 21.1ms; 84% Recall on gate-level in 13.72s
- Fully automated: DFG generation → GCN → Pooling → Readout → MLP, no manual feature engineering



Dataset: Expanded TrustHub — 132 instances (47 HT-Free, 85 HT-Infected). Circuits: AES, RS232, PIC, DES, RC5.

Metric	AES	RS232	PIC	DES	RC5	Avg
RTL Recall	96.3%	95.0%	100%	98.1%	93.5%	96.6%
RTL Precision	97.8%	86.4%	80.0%	98.2%	98.9%	92.3%
RTL F1	96.8%	90.5%	88.9%	98.1%	95.9%	94.0%
Netlist Recall	87.0%	90.0%	77.5%	76.0%	90.0%	84.1%
Det. Time	0.027s	0.017s	0.018s	0.022s	0.022s	21.1ms

ANALYSIS:

- RTL outperforms netlist — behavioral semantics preserved at RTL level are lost after synthesis
- Attention pooling is critical: removing it drops F1 from 94% to 67.7% (ablation study confirms)

RESEARCH GAPS:

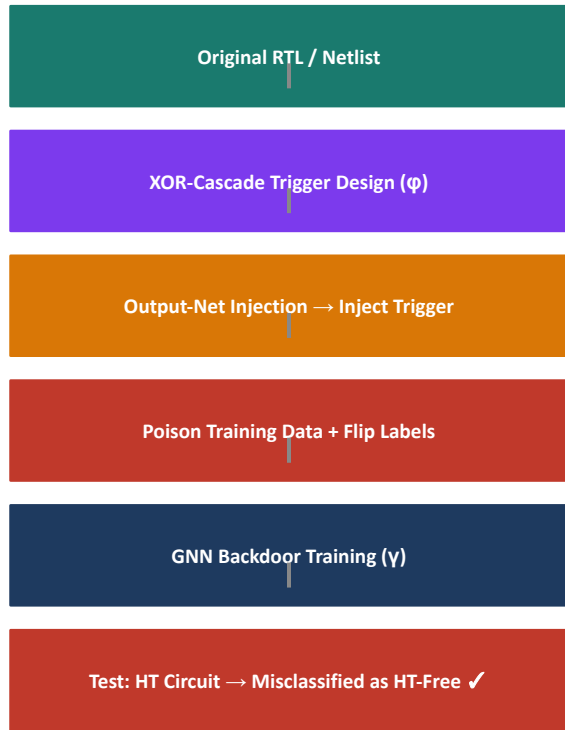
- Purely structural reasoning — RTL semantic content (code intent, signal meaning) never analyzed
- No explainability — cannot identify which gates/signals are Trojan triggers or why detection fired

FUTURE SCOPE:

- Fuse GNN structural embeddings with LLM semantic embeddings via cross-modal attention
- Add GNNExplainer

KEY POINTS:

- First backdoor attack on GNNs in the hardware security context — covers HT hiding and IP piracy
- Existing random subgraph triggers (Erdős–Rényi) violate circuit design rules → cannot be used
- Designs XOR-cascade sub-circuits as backdoor triggers: even inversions → no functional change
- XOR-cascade achieves 100% toggle and statement coverage — bypasses FANCI unused-circuit detection
- Trigger injected on output-connected nets → embedded in DFG root → cannot be removed by synthesis
- Backdoored GNN maintains clean accuracy on normal inputs; misclassifies trigger-embedded circuits
- Attack parameterized by: trigger size ϕ (fraction of nodes) and poisoning intensity γ (training data %)
- Achieves 100% Attack Success Rate on both GNN4TJ (HT detection) and GNN4IP (IP piracy)



Case Study 1 (IP Piracy): GNN4IP — 74 gate-level netlists from ISCAS-85 (c432, c499, c880) + obfuscated variants.

Case Study 2 (HT Hiding): GNN4TJ — AES, PIC, RS232 with TrustHub HTs + DET, RC6, SPI, VGA circuits.

Target Platform	Trigger Size ϕ	Poison Intensity γ	Attack Success Rate	Backdoor Accuracy
GNN4IP (IP Piracy)	2%	60%	100% (ASR-ST)	100%
GNN4IP (IP Piracy)	20%	20%	100% (ASR-ST)	100%
GNN4TJ — AES	30%	25%	100%	~80%
GNN4TJ — PIC	30%	25%	100%	~80%
GNN4TJ — RS232	50%	25%	100%	~80%

ANALYSIS:

- ASR scales with ϕ and γ — MLaaS adversary has no practical upper bound on poisoning intensity
- Backdoor trigger leaves zero footprint post-fabrication: synthesis removes XOR-cascade; only HT payload remains
- Randomized smoothing drops clean GNN4TJ accuracy by 75% yet PoisonedGNN still achieves 100% ASR

RESEARCH GAPS:

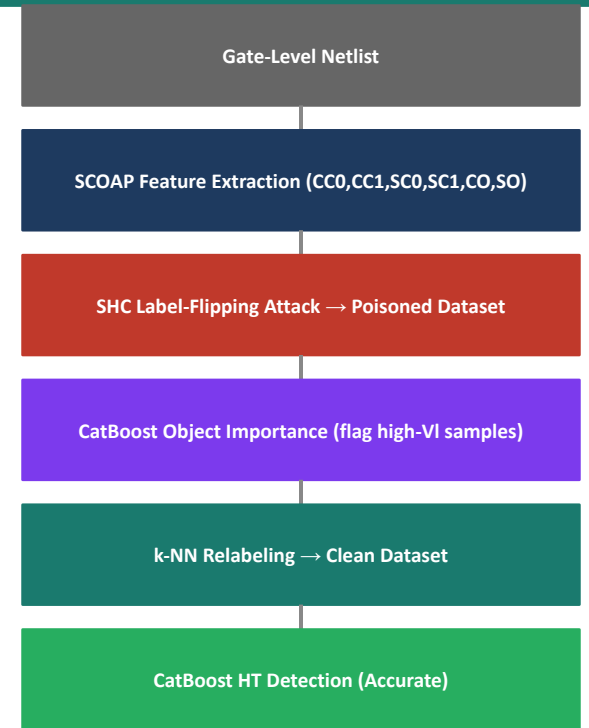
- No defense mechanism proposed — NeuralCleanse and randomized smoothing both fail completely
- Only targets graph-level GNN classifiers; node-wise detection platforms may be inherently resilient
- Only XOR-cascade triggers studied; conditional-statement (branch-based) triggers unexplored

FUTURE SCOPE:

- Hardware-aware backdoor detection using toggle/branch/statement coverage analysis on RTL
- Extend attack to node-wise GNN platforms using a payload-model intercept approach
- Certified defenses with provable guarantees (spectral signatures, randomized ablation)

KEY POINTS:

- Addresses label-flipping poisoning attacks on ML-based HT detection using SCOAP gate-level features
- SCOAP: 6 values (CC0, CC1, SC0, SC1, CO, SO) — Trojan nets have higher controllability/observability
- Proposes Stochastic Hill Climbing (SHC) attack: finds optimal training subset to flip, maximising validation loss
- SHC mutates subset inclusion/exclusion at $\text{pmut}=0.5$; only 20% of optimal subset flipped (stealthiness)
- Defense: CatBoost object importance flags high-loss samples; k-NN relabels them to correct class
- Both attack and defense are black-box / ML model-independent — no knowledge of detection model needed
- Evaluated on Trust-Hub (16 benchmarks) and DeTrust (11 benchmarks)



Trust-Hub: 16 combinational/sequential Trojan benchmarks. DeTrust: 11 modified Trust-Hub circuits (flip-flop at trigger gates). 80% train / 10% validation / 10% test split. SCOAP features via Testability Measurement Tool.

Method	Avg Loss	Accuracy	Recall	FPR	FNR
Clean CatBoost (No Attack)	0.66%	99.87%	97.4%	0.07%	2.5%
After SHC Attack	58.50%	67.09%	50.53%	32.57%	46.16%
After Proposed Defense	1.43%	99.66%	89.45%	0.17%	10.54%
k-NN Defense [Paudice]	5.30%	99.0%	71.46%	0.51%	28.53%
AdaBoost Defense [Cheng]	4.58%	99.28%	62.15%	1.92%	37.84%

ANALYSIS:

- SHC raises average validation loss 25–58% above existing attacks by flipping only the highest-impact 20% of samples
- CatBoost ordered boosting prevents gradient bias from target leakage — more reliable than XGBoost/Random Forest
- Defense identifies ~99% of flipped samples; post-defense accuracy within 0.3% of pre-attack baseline

RESEARCH GAPS:

- Restricted to SCOAP gate-level features — focuses on feature similarity
- Black-box adversary only — white-box attacks with full model access would be far more effective
- No evaluation against adaptive attackers who know the defense (k-NN can be targeted)

FUTURE SCOPE:

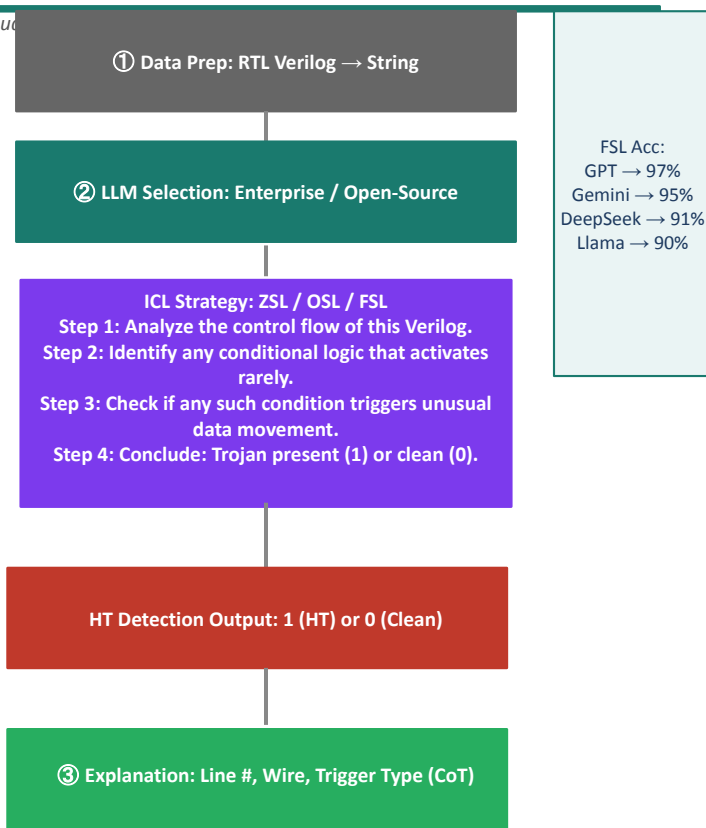
- Extend HT detection to graph-based models (GNNs) to capture circuit structure beyond SCOAP features
- Develop defenses against adaptive attacks using multi-view learning instead of relying only on feature similarity
- Propose a GNN–LLM fusion model combining structural learning and reasoning for robust detection
- Use cross-checking between structure and labels to identify poisoned data

Paper ④ — PEARL: Adaptive & Explainable HT Detection using Large Language Models

[4] R. K. Kundu et al., "PEARL: An Adaptive and Explainable Hardware Trojan Detection Using Open Source and Enterprise Large Language Models," *2024 IEEE/ACM Test Symposium (Test)*, pp. 1–10, 2024.

KEY POINTS:

- First study comparing enterprise (GPT-3.5 Turbo, Gemini-1.5 Pro) vs open-source (Llama-3.1, DeepSeek-V2) LLMs for HT detection
- Uses In-Context Learning (ICL): Zero-shot (ZSL), One-shot (OSL), Few-shot (FSL) on RTL Verilog files directly
- RTL files → string representation → formatted prompt → LLM API or fine-tuned open-source weights
- Chain-of-Thought (CoT) prompting ensures step-by-step structured reasoning for detection and explanation
- After detection, model explains: exact line numbers, wire names, and Trojan trigger type (combinational/sequential)
- Golden-free: no reference design required — directly analyzes unseen benchmark circuits
- FSL GPT-3.5 Turbo: 97% accuracy; DeepSeek-V2: 91% — both outperforming methods
- GPT-3.5 Turbo fine-tunes in 1,680s vs HTNet 44,215s — 26× faster; no golden chips needed



Trust-Hub: AES-T900 to AES-T1700 (8 AES variants, 128-bit I/O). ISCAS'85/'89: c2670, c3540, c5315, c6288, c7552, s13207, s15850, s35932 (up to 16K+ signals, 39,744+ LOC). Trojans: combinational, sequential, hybrid triggers (rare/non-rare signals, sync/async counters).

Model	Trust-Hub Acc	ISCAS Acc	Both Acc	Precision	Recall	F1	Train Time
GPT-3.5 Turbo	100%	94%	97%	1.0	1.0	1.0	1,680s
Gemini-1.5 Pro	95%	90%	96%	0.96	0.96	0.96	1,840s
DeepSeek-V2	93%	85%	91%	0.95	0.94	0.96	9,950s
Llama-3.1 8B	91%	84%	90%	0.94	0.93	0.95	10,800s

ANALYSIS

- FSL > OSL > ZSL across all models — even 10–16 labeled examples dramatically improve detection; enterprise GPT-3.5 outperforms open-source in all settings
- ISCAS-trained models generalise perfectly to Trust-Hub (F1 = 1.0 for 16 shots) — strong cross-circuit-family transfer learning
- LLM explanations correctly name Trojan line numbers and wire names — human-centric interpretability unavailable in any GNN or ML method reviewed

RESEARCH GAPS

- Enterprise APIs (GPT-3.5, Gemini) transmit proprietary RTL to third-party cloud servers — critical IP security and confidentiality risk for industrial designs
- Prompt injection risk — adversarially crafted Verilog comments can mislead LLM reasoning and produce incorrect HT classifications
- Input token limits prevent direct analysis of very large designs (s35932: 39,744+ LOC) — scalability to real industrial-scale ICs is unsolved

FUTURE SCOPE

- LoRA / QLoRA parameter-efficient fine-tuning for local on-premises deployment of Llama-3.1 and DeepSeek-V2 — eliminates cloud privacy risk entirely
- Multi-modal fusion — combine LLM semantic RTL embeddings with GNN structural DFG embeddings via cross-attention mechanism
- RTL profiling and hierarchical summarisation strategies to handle large modern IC designs (RISC-V, GPU accelerators, SoC peripherals)

Objectives

01 To design a detection model that does not fail when RTL obfuscation is used.

02 To enable accurate pinpointing of the exact location of Trojans within a hardware design.

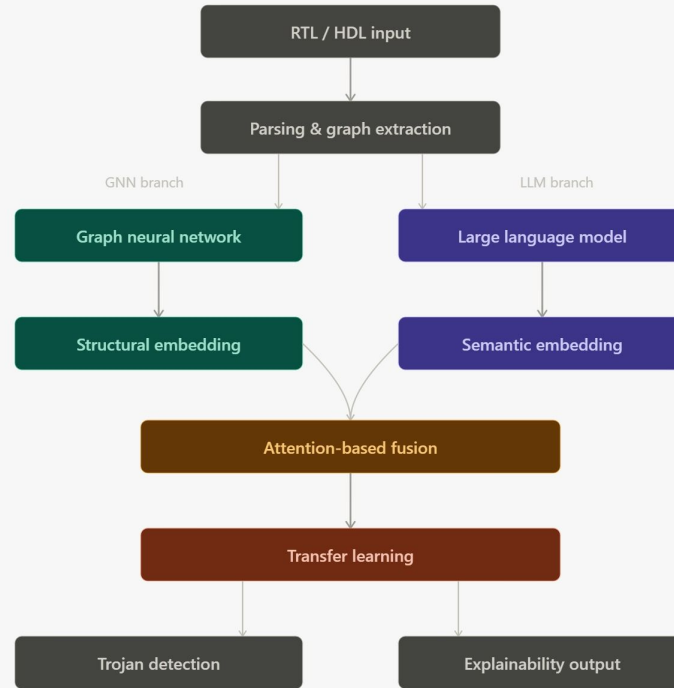
03 To ensure high performance even on unseen and adversarially modified circuits.

04 To provide clear, interpretable decisions that can be traced back to specific hardware components.

Problem Statement

An attention-based fusion model integrating GNN structural reasoning and LLM semantic reasoning, for accurate and explainable pre-silicon Trojan detection.

Proposed Model



System Requirements



Software Requirements

Core ML / DL

Python 3.9+ · PyTorch 2.0+ (CUDA + cuDNN) · PyTorch Geometric / DGL · HuggingFace Transformers

HDL Parsing & Graphs

PyVerilog / hdlparse · Yosys · NetworkX

Simulation & Synthesis

Icarus Verilog · GTKWave · ModelSim / QuestaSim (opt.) · Synopsys DC (opt.)

Data & Tokenization

NumPy · Pandas · scikit-learn · SentencePiece / BPE

Training & Evaluation

TensorBoard · Matplotlib / Seaborn · TorchMetrics · Weights & Biases (opt.)

Dev Tools

Git · VS Code / PyCharm · Anaconda / venv · Jupyter Notebook

LLM APIs (Optional)

OpenAI API (GPT-3.5 / GPT-4) · Anthropic API (Claude) · HF API (Code Llama)



Hardware Requirements

OS

Operating System

Linux (Ubuntu preferred) · macOS · Windows (WSL)

CPU

Processor

Quad-core minimum · 8-core recommended

RAM

Memory

8 GB minimum · 16–32 GB recommended

SSD

Storage

5 GB minimum · 20 GB+ recommended

GPU

GPU (Optional)

NVIDIA with CUDA 11.x + cuDNN

NET

Internet

Stable, high-speed (for LLM API access)

Deliverables

D1**LLM Obfuscation Detection Module**

A fine-tuned LLM pipeline that classifies obfuscated RTL constructs as benign or Trojan-concealing, evaluated with precision, recall, and F1 on obfuscated Trust-HUB benchmarks.

D2**GNN Trojan Localisation Model**

A trained GNN model producing node-level Trojan probability scores on gate netlists and RTL flow graphs, with a heatmap visualisation tool linking flagged nodes to HDL source tokens.

D3**Robustness & Generalisation Evaluation Report**

A comprehensive evaluation report covering ROC curves, confusion matrices, and cross-dataset accuracy across ≥ 3 benchmark suites including obfuscated and unseen RTL variants.

D4**Explainability Tool**

An interactive Explainability tool that maps GNN node activations and LLM attention weights to flagged RTL lines and HDL tokens, demonstrated through case studies on known Trojan circuits.

-
- [1] R. Yasaei, L. Chen, S.-Y. Yu, M. A. Al Faruque, "Hardware Trojan Detection using Graph Neural Networks," IEEE TCAD, arXiv:2204.11431, 2022.
-
- [2] L. Alrahis, S. Patnaik, M. A. Hanif, M. Shafique, O. Sinanoglu, "PoisonedGNN: Backdoor Attack on Graph Neural Networks-Based Hardware Security Systems," IEEE Transactions on Computers, vol. 72, no. 10, pp. 2822–2834, 2023.
-
- [3] R. Sharma, G. K. Sharma, and M. Pattanaik, "A CatBoost based approach to detect label flipping poisoning attack in hardware Trojan detection systems," Journal of Electronic Testing: Theory and Applications, vol. 38, pp. 667–682, Dec. 2022.
-
- [4] R. K. Kundu, K. Khalil, E. Garcia, E. Grassia, P. Calyam, K. A. Hoque, "PEARL: An Adaptive and Explainable Hardware Trojan Detection Using Open Source and Enterprise Large Language Models," IEEE Access, DOI: 10.1109/ACCESS.2025.3592030, 2025.
-
-
-
-
-
-
-
-

Q & A

Thank You

Capstone Project Phase 2 | ISA-2 | Project ID: N56